

System Presentation Reviewer and Script

GrocerEase Inventory Management System

System Presentation Reviewer and Script

System name: GrocerEase Inventory Management System (GrocerEaseIMS)

Group members: Member 1, Member 2, Member 3, Member 4

Purpose: This PDF serves as a reviewer, code guide, and speaking script for understanding and presenting the system during capstone defense.

Important note: This reviewer focuses only on the implemented inventory management features visible in the system code.

System Presentation Reviewer and Script

GrocerEase Inventory Management System

1. System Architecture Overview

GrocerEaseIMS uses a simple PHP and MySQL architecture. The user interacts with PHP views such as dashboard.php, products.php, stock.php, inventory.php, and settings.php. The views display tables, forms, modals, charts, and buttons. When the user submits a form or clicks an AJAX button, the request goes to a PHP controller. The controller validates the request, calls a model class, and the model performs the database query using PDO.

Simple data movement: User/Admin action -> UI page/button/form -> JavaScript/AJAX or HTML form request -> PHP controller -> Model method -> MySQL database -> JSON/HTML redirect response -> UI update or SweetAlert message.

Layer	Main files	Purpose
Frontend/UI	views/inventory/*.php, assets/css/home.css, assets/js/pages.js	Displays pages, tables, forms, modals, buttons, charts, and alerts.
Controller	controllers/products/*.php, controllers/stock/*.php, controllers/backup.php	Receives requests, validates data, calls models, returns redirect or JSON response.
Model	models/product.php, models/Stock/Stocks.php, models/BackupManager.php	Contains database logic and reusable business operations.
Database	config/db.php, MySQL grocer_easedb	Stores products, categories, suppliers, stocks, stock movements, users, logs, and other system records.

2. Member 1 - Reviewer

System Introduction

GrocerEaseIMS is an inventory management system for grocery product control. It helps administrators manage products, categories, suppliers, stock quantities, stock movement logs, dashboard reports, archived records, and backup/recovery.

Problem Statement

Manual inventory tracking can lead to inaccurate product records, delayed stock updates, missing supplier information, and difficulty monitoring stock status. The system was created to organize these records in one place and reduce manual mistakes.

Objectives

- Maintain accurate product, category, supplier, stock, and movement records.
- Allow administrators to add, update, archive, restore, and permanently remove records where implemented.
- Provide dashboard visibility for products, categories, suppliers, low stock, out of stock, and reports.

System Presentation Reviewer and Script

GrocerEase Inventory Management System

- Protect database records through backup and recovery.

Users, Scope, and Limitations

- Primary user: authenticated administrator or inventory staff.
- Scope: inventory management, admin dashboard, product/category/supplier management, stock control, logs, archived records, and backup/recovery.
- Limitation: the current system focuses on internal inventory operations for authorized users.

Key Points to Memorize

- The system is built with PHP, MySQL, PDO, JavaScript, CSS, and SweetAlert.
- The main value is organized inventory data and safer database recovery.
- Most modules follow View -> Controller -> Model -> Database.

Member 1 Q&A

Q: What is the purpose of GrocerEaseIMS?

A: To help manage grocery inventory records, stocks, suppliers, reports, and backup/recovery in one system.

Q: Who are the users?

A: The users are administrators or authorized inventory staff.

Q: What is the main limitation?

A: It is focused on internal inventory operations and authorized system management.

3. Member 1 - Presentation Script

Good day everyone. We are presenting GrocerEase Inventory Management System, or GrocerEaseIMS. This system was created to help organize grocery inventory records such as products, categories, suppliers, stock levels, and inventory movement logs.

The problem we want to solve is the difficulty of manually tracking grocery inventory. Manual records can become inaccurate, delayed, or hard to monitor, especially when products, suppliers, and stock changes increase.

Our objectives are to provide organized inventory management, support admin CRUD operations, monitor stock levels through the dashboard, record inventory movements, and protect the database using backup and recovery. The target users are administrators or authorized inventory staff.

I will now pass the discussion to Member 2, who will explain the user flow and operational transaction flow of the system.

4. Member 2 - Reviewer

User/Operational Side Features

For this system, Member 2 should explain the authenticated user flow and operational transaction flow, especially stock-in and stock-out because these are the transaction

System Presentation Reviewer and Script

GrocerEase Inventory Management System

operations implemented in the code.

- Login/authentication protects inventory pages through includes/auth_check.php.
- Stock Control lets authorized users update product quantity through Stock In and Stock Out.
- Inventory Logs show stock movement records with reference IDs, quantity, reason, date, and product name.
- Validation prevents empty or invalid quantities. Stock Out also prevents subtracting more than current stock.

Important Files and Functions

File	Important code	Purpose

controllers/stock/stock_in_pro	Stocks::StockIn, StockMovements::AddStockMovemen	Adds quantity and records an IN movement.
controllers/stock/stock_out_pr	Stocks::StockOut, GetQuantityByProductId	Subtracts quantity only if enough stock exists.
models/Stock/Stocks.php	StockIn, StockOut, GetTotalLowStockItems	Database operations for stock data.
models/Stock/stockmovements.ph	AddStockMovements, GetInventoryLogsTrend	Records and reports stock movement data.

Sample Code Snippet

```
File: controllers/stock/stock_out_process.php
elseif ($quantity > $currentStock['quantity'])
    $error['quantity'] = 'Insufficient stock. Current stock: ' . $currentStock['quantity'];

$result = $stock->StockOut($product_id, $quantity, $date);
$movement_result = $stock_movement->AddStockMovements($quantity, "OUT", $reference_id, $reason,
$date_movement, $product_id);
```

This code first checks if the requested stock-out quantity is greater than available stock. If it is too high, the process stops and shows an error. If valid, the system subtracts the quantity and records the movement as OUT.

Member 2 Q&A

Q: What transaction flow is implemented?

A: The implemented transaction flow is stock movement, specifically Stock In and Stock Out.

Q: How does Stock Out prevent negative stock?

A: The controller checks current stock and the SQL update also requires quantity >= requested quantity.

Q: What is recorded after a stock transaction?

A: The system records a stock movement with quantity, reference type, reference ID, reason, date, and product ID.

5. Member 2 - Presentation Script

System Presentation Reviewer and Script

GrocerEase Inventory Management System

For my part, I will explain the operational user flow. The main transaction flow is inventory movement, specifically stock in and stock out.

When an authorized user performs Stock In, the system validates the quantity, adds it to the current stock, and records the movement with a reference ID. When the user performs Stock Out, the system checks if the requested quantity is valid and if there is enough stock. This prevents negative inventory.

Behind the system, the controller receives the form request, calls the Stocks model, updates the database, then records the movement in the stock_movements table. This helps the admin trace why the stock changed. I will now pass the presentation to Member 3 for the admin dashboard and management side.

6. Member 3 - Reviewer

Admin Dashboard and Management

The admin side is the main control center of GrocerEaseIMS. The dashboard shows key performance indicators and charts. Admin modules manage products, categories, suppliers, stock, inventory logs, archives, and backup/recovery.

- Dashboard KPIs: active products, categories, suppliers, total stocks, low stock, out of stock.
- Reports/charts: stock per category, stock status, and inventory trend from stock_movements.
- CRUD modules: products, categories, suppliers, product suppliers, stock control.
- Maintenance: archived records and backup/recovery.

Important Code Snippets

```
File: views/inventory/dashboard.php
$total_product = $product->GetTotalProducts();
$total_categories = $category->GetTotalCategories();
$total_suppliers = $supplier->GetTotalSupplier();
$total_quantity = $stock->GetTotalStockQuantity();
$low_stock = $stock->GetTotalLowStockItems();
$out_of_stock = $stock->GetTotalOutOfStockItems();
```

The dashboard creates model objects and asks each model for totals. The returned numbers are displayed as dashboard cards. Chart data comes from model queries and is passed into JavaScript Chart.js using json_encode.

Admin Database Tables

- products: stores product information and delete/status flags.
- categories: stores product category information.
- suppliers: stores supplier information.
- stocks: stores current stock quantity per product.
- stock_movements: records stock in/out history and chart data.
- transaction_logs: logs quantity changes for stock operations.

System Presentation Reviewer and Script

GrocerEase Inventory Management System

Member 3 Q&A

Q: Why is the dashboard important?

A: It gives quick visibility of product count, supplier count, stock levels, and stock problems.

Q: What does CRUD mean?

A: Create, Read, Update, and Delete records.

Q: How do reports help management?

A: They help the admin see stock status and inventory trends for decision-making.

7. Member 3 - Presentation Script

For the admin side, the dashboard acts as the control center of the system. It shows important numbers such as active products, total categories, total suppliers, total stocks, low stock items, and out of stock items.

The dashboard also has charts for stock overview, stock status, and inventory trends. These charts help the admin understand what is happening in the inventory without manually checking every record.

Admin management features include adding, editing, deleting, and restoring records depending on the module. The system uses models to run database queries and controllers to process admin actions. These features help maintain accurate inventory data. I will now pass the discussion to Member 4 for the Product Module and Backup and Recovery Module.

8. Member 4 Reviewer - Product Module

Purpose and Importance

The Product Module manages grocery products. It is important because products are the center of the inventory system. Stock records, supplier relationships, categories, and reports all depend on accurate product data.

- Access: authenticated admin/inventory staff only.
- Main page: `views/inventory/products.php`.
- Main model: `models/product.php`.
- Main controllers: `add_product_process.php`, `edit_process.php`, `delete_product.php`, `recover.php`, `hard_delete_product.php`.

Implemented Product Features

- Add product: creates a product and also creates an initial stock row with quantity 0.
- View product list: `products.php` displays active products with category, price, description, supplier, and status.
- Edit product: updates product name, category, selling price, description, and status.
- Delete product: soft deletes by setting `is_deleted = 1` and `deleted_at = NOW()`.
- Search product: searches product names by prefix using `LIKE search%`.
- Pagination: products are displayed with page and limit.

System Presentation Reviewer and Script

GrocerEase Inventory Management System

- Status: active/inactive status controls whether manage supplier is enabled.

Product Table Fields Visible in Code

- product_id_pk: primary key.
- product_name: product name.
- category_id_fk: linked category.
- selling_price: selling price.
- product_description: product description.
- status: active/inactive flag.
- is_deleted and deleted_at: archive/soft-delete tracking.

Add Product Code and Explanation

```
File: controllers/products/add_product_process.php
$product_name = ucfirst(trim($_POST['product_name'] ?? ''));
if (empty($product_name) || empty($category) || empty($selling_price) || $status === '')
    $errors['form'] = "Please fill in all required fields.";
if ($product->CheckDuplicateProduct($product_name, $category))
    $errors['product_name'] = "Product already exists in the selected category.";
$product_id = $product->AddProduct($product_name, $category, $selling_price, $description,
$status);
$stock->AddProductStock($product_id, 0, date('Y-m-d H:i:s'));
```

The controller reads POST data, validates required fields, checks duplicates, sanitizes input, inserts the product, and then creates a stock record with quantity 0. This means every product automatically has a matching row in stocks.

```
File: models/product.php
public function AddProduct($name, $category, $selling_price, $description, $status)
{
    $stmt = $this->conn->prepare("INSERT INTO products
(product_name,category_id_fk,selling_price,product_description,status)
VALUES(:name,:category,:selling_price,:description,:status)");
    $stmt->execute([...]);
    return $this->conn->lastInsertId();
}
```

The model uses a prepared statement, which is safer than directly inserting values into SQL. After insert, it returns the new product ID so the stock table can link to it.

Edit Product Logic

```
File: controllers/products/edit_process.php
$original = $product->GetProductInfoById($product_id);
if ($product_name != $original['product_name'] || $category != $original['category_id_fk'])
    if ($product->CheckDuplicateProduct($product_name, $category))
        $errors['product_name'] = "Product already exists in the selected category.";
$result = $product->UpdateProductInfo($product_id, $product_name, $category, $selling_price,
$description, $status);
```

The edit process compares the submitted data with the original record. If the name or category changed, it checks for duplicates. Then the model updates the product fields.

Delete Product Logic

System Presentation Reviewer and Script

GrocerEase Inventory Management System

```
File: models/product.php
public function SoftDeleteProduct($id)
{
    $stmt = $this->conn->prepare("UPDATE products SET is_deleted = 1,deleted_at = NOW() WHERE
product_id_pk = :id");
    $stmt->execute([':id' => $id]);
    return $stmt->rowCount() > 0;
}
```

Delete is a soft delete. The record remains in the database but is hidden from the active product list. This supports archive and restore behavior.

Display and Search Logic

```
File: views/inventory/products.php
if (!empty($search)) {
    $products = $product->SearchProduct($search);
} else {
    $products = $product->GetPaginatedProducts($page, $limit);
}
```

The view decides whether to show search results or paginated product results. The product model joins products with categories, preferred supplier data, and supplier name.

Validation, Error Handling, and Concerns

- Product name, category, selling price, and status are required.
- Product name must not exceed 100 characters.
- Selling price must be numeric and positive.
- Duplicate product names are blocked within the same category.
- Errors are stored in session and displayed after redirect.
- Possible improvement: use stricter numeric input type and CSRF tokens.
- No product image upload is implemented in the visible product code.

9. Member 4 Reviewer - Backup and Recovery Module

Purpose and Importance

Backup and Recovery protects the database. It allows the admin to create a full SQL backup, restore a valid backup, view backup history, download backup files, and delete backup files. Restore must be restricted to admin because it can overwrite current database data.

- Main UI: views/inventory/settings.php.
- Main controller: controllers/backup.php.
- Main model: models/BackupManager.php.
- Storage: local /backups folder.
- Automatic Backup note: cron_backup.php exists as backend utility, but scheduled backup management is not presented as a user-facing module in the Backup and Recovery UI.

Backup Flow

1. Admin opens Backup and Recovery module.

System Presentation Reviewer and Script

GrocerEase Inventory Management System

2. Admin clicks Backup now.
3. System checks login/session and request method.
4. BackupManager reads all MySQL base tables.
5. System exports table structure and data into a SQL file.
6. System saves the file in /backups.
7. System validates the backup file and returns success or failure JSON.
8. The UI shows SweetAlert and refreshes backup history.

Restore Flow

1. Admin uploads or selects a SQL backup file.
2. System validates file extension, readability, size, and SQL-like content.
3. Admin confirms restore warning.
4. System creates a pre_restore safety backup before changing data.
5. System disables foreign key checks, executes SQL statements, and enables checks again.
6. System returns success or failure with safety backup information.

Backup History Flow

1. UI calls controllers/backup.php?action=list.
2. BackupManager scans /backups/*.sql files.
3. System returns filename, date, type, size, readable size, and valid status.
4. Admin can download, restore, or delete selected backup.

Decision Points

- Is database connection valid? The DB connection is created in autoload.php through config/db.php.
- Was backup created successfully? The SQL file must be written and validated.
- Does admin confirm restore? SweetAlert confirmation is required before restore.
- Is selected backup valid? validateBackupFile checks extension, readability, size, and content.
- Was restore completed successfully? SQL statements must execute without exception.
- Admin chooses backup history action: download, restore, or delete.

Backup Creation Code

```
File: models/BackupManager.php
$tables = $this->getTables();
$this->writeLine($handle, 'SET FOREIGN_KEY_CHECKS=0;');
foreach ($tables as $table) {
    $this->dumpTable($handle, $table);
}
$this->writeLine($handle, 'SET FOREIGN_KEY_CHECKS=1;');
$validation = $this->validateBackupFile($filepath);
```

The backup process gets all database tables, writes SQL settings, dumps every table structure and data, re-enables foreign key checks, then validates the generated SQL file.

Restore Code

System Presentation Reviewer and Script

GrocerEase Inventory Management System

```
File: models/BackupManager.php
$validation = $this->validateBackupFile($filepath, $sourceName);
$safetyBackup = $this->createBackup('pre_restore');
$statements = $this->splitSqlStatements($sql);
$this->conn->exec('SET FOREIGN_KEY_CHECKS=0');
foreach ($statements as $statement) { $this->conn->exec(trim($statement)); }
$this->conn->exec('SET FOREIGN_KEY_CHECKS=1');
```

Before restore, the system validates the selected file and creates a safety backup. The safety backup is a copy of the current state before restore, not necessarily the original good state. This is useful if the restore file was wrong.

Controller Actions

Action	Method	Result
backup	POST	Creates full SQL backup and returns JSON.
restore	POST	Restores uploaded SQL file.
restore_specific	POST	Restores selected history file.
validate	POST	Validates uploaded or selected SQL file.
list	GET	Returns backup history JSON.
download	GET	Streams SQL file to browser.
delete	POST	Deletes selected backup file after path validation.

Security Review

- Controller requires logged-in session.
- Request methods are restricted for important actions.
- Only .sql files are accepted for restore.
- resolveBackupPath validates filenames with basename, regex, realpath, and folder containment.
- Delete requires confirmation in the UI and POST in the controller.
- Restore shows warning because it can overwrite database data.
- Recommended improvement: protect /backups from direct public access and add CSRF tokens.

Common Errors and Fixes

Error	Cause	Fix
404 Not Found	Wrong controller path or missing file.	Check relative path ../../controllers/backup.php.
Unexpected token <	HTML returned instead of JSON, usually login page, 404, or PHP error.	Check session, controller path, and PHP warnings.
History not loading	action=list failed or JSON invalid.	Check controller response and browser console.
Restore button disabled	No valid SQL file selected or validation failed.	Use valid .sql backup generated by the system.

System Presentation Reviewer and Script

GrocerEase Inventory Management System

Invalid SQL file	File is empty, unreadable, wrong	Choose a readable .sql backup file.
	extension, or not SQL-like.	

10. Member 4 - Presentation Script

Part 1: Product Module Script

First, I will explain the Product Module. This module is important because products are the main records of the inventory system. The admin can add, view, edit, delete, and search products.

In this part, the admin can click Add Product and enter the product name, category, selling price, description, and status. Behind the system, the form sends a POST request to `controllers/products/add_product_process.php`. The controller validates required fields, checks if the product already exists in the selected category, and checks if the selling price is numeric and positive.

After validation, the controller calls the Product model. The model inserts the product into the products table using a prepared SQL statement. After that, the system also creates a stock record with zero quantity, so every new product is ready for stock control.

For editing, the controller gets the original product data and compares it with the submitted data. If the product name or category changes, it checks for duplicates again. Then it updates the products table.

For delete, the system uses soft delete. It does not immediately remove the product from the database. Instead, it sets `is_deleted` to 1 and records `deleted_at`. This is important because the system can still manage archived records and avoid accidental permanent loss.

This module is important because accurate product records affect stock management, suppliers, reports, and backup demonstration.

Part 2: Backup and Recovery Script

Next, I will explain the Backup and Recovery Module. This module protects the system database. The admin can create a backup, restore a valid backup, view backup history, download backup files, and delete old backup files.

When the admin clicks Backup now, the button sends a POST request to `controllers/backup.php?action=backup`. The controller checks if the user is logged in and then calls `BackupManager`. `BackupManager` reads all database tables, writes table structures and data into a SQL file, saves it in the backups folder, validates the file, and returns a JSON response.

For restore, the admin must upload or select a valid SQL backup file. The system validates the file first. Then the admin confirms the restore warning. Behind the system, `BackupManager` creates a `pre_restore` safety backup before applying the selected backup. This is important because restore can overwrite current database data.

Backup History loads using `controllers/backup.php?action=list`. The system scans the backups folder and shows the filename, date, type, size, and status. The admin can download, restore, or delete each backup. Download streams the file, restore validates and applies it,

System Presentation Reviewer and Script

GrocerEase Inventory Management System

and delete removes the selected backup after confirmation.

Automatic Backup is not presented as part of the user-facing module because the Backup and Recovery page focuses on manual backup and restore actions. The important implemented features for this presentation are Create Backup, Restore Backup, View Backup History, Download Backup, and Delete Backup.

This module helps protect the system because if product, stock, or supplier records are accidentally changed, a valid backup can restore the database to a previous state. That completes my part. Thank you.

11. Code Reviewer Section

Module improvement	Snippet purpose	Related table	Issue /
Product Add	AddProduct inserts product and creates stock row with quantity 0.	products, stocks	Add CSRF token and stricter numeric input.
Product Update	UpdateProductInfo edits product fields after validation.	products	Variable typo \$error/\$errors exists but PHP still uses \$errors dynamically. Clean naming.
Product Delete	SoftDeleteProduct sets is_deleted = 1.	products	Good for archive; permanent delete must be controlled.
Product Search	SearchProduct uses product_name LIKE search%.	products	Search is prefix-based only. Could support wider search.
Backup Create	createBackup writes table SQL to .sql file.	all base tables	Protect /backups folder from public access.
Restore	restoreBackup validates file, creates safety backup, executes SQL.	all restored tables	Restore is risky; admin and CSRF protection recommended.
History List	getBackupList scans local .sql files.	file system	No persistent audit table.
Download/Delete	resolveBackupPath protects file path.	file system	Good path validation; still protect directory at server level.

12. System Flow Reviewer

Flow Output	User/Admin action	System process	Database action	
Login	User enters credentials.	Authentication checks account.	Reads users table. 	Dashboard if val.
Admin Dashboard	Admin opens dashboard. 	Models collect counts and chart data. 	Reads products, categories, suppliers, stocks, movements. 	Cards and charts display.
Product Add	Admin submits add form.	Controller validates and calls Product model.	INSERT products; INSERT stocks.	Redirect with success/error.
Product Delete	Admin confirms delete.	Controller calls	UPDATE products SET	Product hidden f

System Presentation Reviewer and Script

GrocerEase Inventory Management System

			SoftDeleteProduct.		is_deleted=1.		active list.	
Backup		Admin clicks Backup		BackupManager exports		Reads all base tables.		SQL backup file
		now.		tables.				
Restore		Admin selects SQL and		Validate, safety backup,		Tables are restored from		Success/error al
		confirms.		execute SQL.		SQL.		
History		Admin opens backup		Controller lists files.		No DB table; file scan		Table of backups
		history.				only.		

13. Possible Panel Questions and Answers

Member 1 Q&A

Q: What problem does the system solve?

A: It organizes inventory records and reduces manual tracking errors.

Q: What is included in the scope?

A: Admin inventory management, dashboard, product/category/supplier records, stock control, logs, archives, and backup/recovery.

Q: What is the scope of Member 1?

A: System purpose, users, objectives, scope, limitations, and the overall inventory workflow.

Member 2 Q&A

Q: What is the transaction flow?

A: The implemented transaction flow is Stock In and Stock Out.

Q: How is validation handled?

A: The controllers check required fields, positive quantity, and sufficient stock before updating records.

Member 3 Q&A

Q: How does the dashboard help?

A: It summarizes important inventory information and visualizes stock status and trends.

Q: What are maintenance features?

A: Archived records and backup/recovery help maintain and protect system data.

Member 4 Q&A - Product

Q: What is the purpose of the Product Module?

A: To manage grocery product records used by stock, suppliers, and reports.

Q: How do you add a product?

A: The admin submits the add form, the controller validates it, then the Product model inserts the record.

Q: What happens when a product is deleted?

A: It is soft deleted by setting is_deleted to 1, so it is hidden but not immediately removed.

System Presentation Reviewer and Script

GrocerEase Inventory Management System

Q: What validations are used?

A: Required fields, duplicate check, product name length, numeric and positive selling price.

Q: What table stores product data?

A: The products table stores product information.

Member 4 Q&A - Backup and Recovery

Q: What is the purpose of backup and recovery?

A: To protect system data by saving and restoring database state.

Q: Where are backup files stored?

A: They are stored as .sql files in the local /backups folder.

Q: What happens during restore?

A: The file is validated, a safety backup is created, and SQL statements are executed into the database.

Q: Why is restore risky?

A: It can overwrite current database records.

Q: How are backup files validated?

A: The system checks .sql extension, readability, non-empty size, and SQL-like content.

Q: Why is Automatic Backup not included in the presentation module?

A: Because the user-facing Backup and Recovery page focuses on manual backup and restore actions. The presented implemented features are create, restore, history, download, and delete.

Q: What if backup fails?

A: The controller returns error JSON and the UI shows a failure alert.

Q: What if restore fails?

A: The system returns an error message and includes the safety backup filename if one was created.

Q: How do you protect backup files?

A: Use login protection, validated paths, and recommended server rules to prevent public access to /backups.

14. Final Group Presentation Flow

1. Member 1 opens with greeting, system introduction, problem statement, objectives, users, scope, and overall modules.
2. Transition: "I will now pass the discussion to Member 2 for the user and transaction flow."
3. Member 2 explains authenticated user flow, stock in/out transaction flow, validation, and movement logs.
4. Transition: "Next, Member 3 will explain the admin dashboard and management features."
5. Member 3 explains dashboard, reports, CRUD management, records, and maintenance.

System Presentation Reviewer and Script

GrocerEase Inventory Management System

6. Transition: "Now Member 4 will discuss the Product Module and Backup and Recovery."
7. Member 4 explains Product Module in detail, then Backup and Recovery in detail.
8. Final closing: "Overall, GrocerEaseIMS helps manage grocery inventory records, monitor stock, and protect data through backup and recovery. Thank you."

15. Final Summary

GrocerEaseIMS is a PHP and MySQL inventory management system. The modules work together by managing product records, linking categories and suppliers, updating stock quantities, recording inventory movements, showing dashboard reports, and protecting records through backup and recovery.

Member 1 should focus on system purpose, objectives, users, and scope. Member 2 should focus on authenticated user flow and stock transaction flow. Member 3 should focus on dashboard, reports, CRUD, and maintenance. Member 4 should focus strongly on Product Module and Backup and Recovery because these parts include important code logic, database operations, validation, and system protection.